12.5 Example: Reading a Data File

Suppose you had a data file consisting of rows and columns of numbers:

```
1       2       34
5       6       78
9       10      112
```

Suppose you wanted to read these numbers into an array. (Actually, the array will be an array of arrays, or a ``multidimensional'' array; see section 4.1.2.) We can write code to do this by putting together several pieces: the `fgetline` function we just showed, and the `getwords` function from chapter 10. Assuming that the data file is named `input.dat`, the code would look like this:

```
#define MAXLINE 100
#define MAXROWS 10
#define MAXCOLS 10

int array[MAXROWS][MAXCOLS];
char *filename = "input.dat";
FILE *ifp;
char line[MAXLINE];
char *words[MAXCOLS];
int nrows = 0;
int n;
int i;

ifp = fopen(filename, "r");
if(ifp == NULL)
        {
        fprintf(stderr, "can't open %s\n", filename);
        exit(EXIT_FAILURE);
        }

while(fgetline(ifp, line, MAXLINE) != EOF)
        {
        if(nrows >= MAXROWS)
                {
                fprintf(stderr, "too many rows\n");
                exit(EXIT_FAILURE);
                }

        n = getwords(line, words, MAXCOLS);

        for(i = 0; i < n; i++)
                array[nrows][i] = atoi(words[i]);
        nrows++;
        }
```

Each trip through the loop reads one line from the file, using `fgetline`. Each line is broken up into ``words'' using `getwords`; each ``word'' is actually one number. The numbers are however still represented as strings, so each one is converted to an `int` by calling `atoi` before being stored in the array. The code checks for two different error conditions (failure to open the input file, and too many lines in the input file) and if one of these conditions occurs, it prints an error message, and exits. The `exit` function is a Standard library function which terminates your program. It is declared in `<stdlib.h>`, and accepts one argument, which will be the *exit status* of the program. `EXIT_FAILURE` is a code, also defined by `<stdlib.h>`, which indicates that the program failed. Success is indicated by a code of `EXIT_SUCCESS`, or simply 0. (These values can also be returned from `main()`; calling `exit` with a particular status value is essentially equivalent to returning that same status value from `main`.)

Read sequentially: prev next up top